

Personal Finance Dashboard

Conclusion & Reflection (Fazit & Reflexion)

Roman Lysser · Module WEBLAB.F2601 · 25. September 2026

1. What Went Well?

- **Full scope on time.** All 21 Must and 7 Should stories are implemented, in 63 hours against a target of about 60. The MoSCoW list with an explicit, reasoned Won't-have section made it easy to say no; Could stories were not started, apart from real sign-in, which replaced the simulated login early.
- **Working with an AI agent.** The coding agent was surprisingly capable. Once I split the work into small features, it produced working slices quickly, while I kept the overview of the codebase and made every decision myself. AGENTS.md files carried the project conventions into each session. The 63 journal hours are my own working time.
- **Feature-first structure.** Backend features share the same four layers (api, model, repository, service), frontend features keep their pages, components, and services together. New features such as budgets fitted in without touching unrelated code.
- **Decisions written down.** Six ADRs record the non-obvious choices, including two deviations from my own proposal ("Uncategorized" as a null category, month-to-month limit take-over).
- **Tests on every level.** 191 backend unit, 111 API integration, and 277 frontend unit tests, plus 8 Playwright flows on desktop and mobile. Coverage is 76–90 % of statements. The integration suite runs the real application against an in-memory database, without mocks.
- **Quality targets met.** Lighthouse averages of 100 (desktop) and 96.5 (mobile) on the recorded pages. Mobile details were taken seriously: the shell keeps content above the iOS Firefox toolbar, and the add dialog opens with the keyboard already up.
- **Production defaults.** A required auth secret, Swagger only outside production, a Content Security Policy, self-hosted fonts, and rate limiting on the real client IP.

2. Where Were the Challenges?

- **Pacing the AI.** At the start, the agent tried to build the whole app in one go, which was not the goal. I set up the project by hand and then worked feature by feature, asking for one small change at a time and reviewing it before the next. Keeping the agent from doing more than I asked took constant attention. Skills, reusable instructions for specific tasks, were useful to guide the agent where general prompts were not enough.
- **Currency detour.** I built multi-currency support with mirrored ECB exchange rates (13 Sept) and removed it three days later, although the proposal had already excluded it (W6). About four hours went into code that no longer exists.
- **Tests and documentation came late.** Most integration tests, all Playwright tests, and the arc42 documentation were written in the last four days. A final review then found issues that earlier tests would have caught: budget limits and their month marker were written in two

separate statements, and the integration suite failed about one run in four because of a port collision with other apps on macOS.

- **Uneven pace.** Work happened on 15 of 25 days, with gaps of up to three days and 12 commits on the last day.
- **Security hardening late.** The required secret, CSP, and rate-limit IP handling were added on 22 Sept instead of being part of the Compose setup from the start.
- **Proposal versus reality.** The proposal assumed a reserved catch-all category, previous-month limits, and one-command start. Each turned out to need a decision (ADR-1, ADR-2, ADR-6), and C1 remains only partly done: sign-up works through the API, but there is no sign-up page.

3. What Would I Do Differently or Better Next Time?

- **Brief the AI in small steps from day one.** Write the conventions and a short task list before the first prompt, and give the agent one feature at a time, instead of reining it in after it overshoots.
- **Check the Won't-have list before starting a feature.** A two-minute check would have saved the exchange-rate detour.
- **Write the integration test with the feature.** Especially for multi-step writes: a fault-injection test in the same commit proves atomicity instead of assuming it.
- **Keep the architecture documentation growing with the code.** Add an ADR on the day of the decision and draw the diagram with the feature, not in the last week.
- **Set up production defaults on day one.** Secret handling, CSP, and a production Compose run belong to the first deployable version.
- **Spread the work evenly and keep commits small.** Several commits mix features (for example, overview stats with a restyle); one concern per commit makes review and rollback easier.
- **Make the start really one command.** Generate and persist the auth secret on first start, so M20 holds without a manual setup step.